Chap 2 (1) Data and Sampling Distributions -

PAGE NO.
DATE / /

1. (a) Random Sampling and Sample Bias

(1) Key Definitions

- **Population**
  The complete (often theoretical) dataset of interest.

- **Sample**
  A subset of the population used for analysis

- **N (n)**
  Population size (sample size)

- **Random Sampling**: Every population member has an equal chance of selection.
  - ↳ with replacement: observations can be resampled (e.g. bootstrapping).
  - ↳ without replacement: observation are unique in the sample.
- **Stratified Sampling**: Population divided into sub groups (Strata), sampled separately for better.

(2) Sample Bias

- Occurs when a sample systematically mis represent the population.

- **Causes**
  - ↳ Non-random selection (e.g. Literary Digest Poll, 1936).
  - ↳ self-selection (e.g. Yelp reviews → every extreme opinions).

- **Impact**
  - ↳ Biased estimates, unreliable models

3) Data Quality > Quantity

↳ Representativeness matters more than sheer size (e.g. Gallup's 2,000 polls vs. Literacy Digest 10M biased ones).

↳ common Pitfalls

↳ Convenience sampling: using easily available data.

↳ Survivorship Bias: ignoring "invisible" data. (e.g. failed product, churned users).

---

4) Practical Implications

↳ For Data Scientist

↳ prefer random or stratified sampling over convenience sample

↳ Audit data for bias sources (e.g. socio economic, skew, self-selections)

↳ Use domain knowledge to define strata (e.g. age groups, regions).

"Let's try to understand with simple explanation"

1) What is sampling

→ Imagine you have a big bowl of soup (population). You can't drink it all. You take a spoonfull (sample) to taste it.

- Good Sample: Spoon mixes everything well → tastes like the whole bowl
- Bad Sample: Only scoops corners → Misleads you about the soup.

PAGE NO.....
DATE / /

2) Random Sampling
↳ Goal: Every "ingredient" (data point) has a fair chance to be picked.
↳ Like: Stirring the soup well before scooping

3) Sample Bias
↳ What? Your spoon picks only carrots (ignoring meat, broth).
↳ Real Examples:
↳ Literary Digest Poll (1936): Only asked rich people → predicted election wrong.
↳ Yelp Review → Only angry/happy people write → skews restaurant ratings.

4) How to Avoid Bias?
↳ Random Sampling: Use a lottery system (e.g. population.sample() in Python).
↳ Stratified Sampling: Ensure each group (e.g. age, income) is fairly represented.

Python Examples (1) Random vs Biased.

PAGE NO.....
DATE / /

```
import pandas as pd.
import numpy as np.

# Simulate a population with age and spending habits. (young spend less).
(1) np.random.seed(42)
population = pd.DataFrame({ (2)
    'age': np.random.randint(18, 70, size=10000),
    'spends': np.random.normal(loc=50, scale=20, size=10000)
3) (3)
population.loc[population['age']<30, 'spends'] *= 0.7
                                    # Young spend 30% less.

# -- Random Sample (unbiased) --.
random_sample = population.sample(n=200, replace=False).
(4) print("Random Sample Mean Spend:", random_sample['spends'].
        mean()).
# --- Biased Sample (only older adults, like Literacy Digest) --.
(5) biased_sample = population[population['age']>40].sample
        (n=200).
print("Biased Sample Mean Spend:", biased_sample['spends'].
        mean())

# Stratified sample (age groups)
(6) strata = pd.cut(population['age'], bins=[18,30,50,70]).
    stratified_sample = population.groupby(strata, group_keys=False)
        .apply(lambda x: x.sample(n=50)).
print("Stratified Sample Mean Spend:", stratified_sample
        ['spends'].mean()).
```

(1) np.random.seed(42) & (2) np.random.randint(18, 70, size = 10000).

(1) what is does!

→ It "locks" Python's random number generator to produce the same sequence of "random" numbers every time you run your code.

→ Seed(42) → The value doesn't matter - only that you use the same seed to get the same result.

- ↳ without seed(42), these numbers would change every time you run the code
- ↳ with seed(42), they stay the same forever.

→ Why 42?

→ It's just a popular number in programming (from the Hitchhiker's Guide to the Galaxy where "42" is the "Answer to the Ultimate Question of Life".

(2) np.random.randint(18, 17, size = 10000).

→ what it does: Generates random integers between a range.

→ Syntax

np.random.randint(low, high)

→ Why is this useful?

- ↳ Simulating dice rolls (randint(1, 7) → 1 to 6).
- ↳ Randomly picking ages (randint(18, 70) → 18 to 69).
- ↳ Anywhere you need unpredictable (but controllable) numbers.

(3) Combined them

Concept	what it does	why it Matters.
seed(42)	locks randomness	Makes "random" results repeatable.
randint()	Generates random integers	Simulates real-world uncertainty (age, dice etc.
Combined	Controlled randomness	Debugging + fair comparisons.

PAGE NO.....
DATE / /

(4) randint(low, high, size=).

b) size: the size parameter controls how many random numbers to generate.

Example: The Candy Jar Example!

(a) ~~whi~~ without out size (picking one candy):

np.random.randint(18, 70) # Picks ONE random number

→ Got N numbers of in a 1D

(b) with size: 5 (picking 5 candies).

np.random.randint(18, 70, size=5) # Picks 5 numbers

(e.g. [42, 35, 27, 58, 19]).

(c) with size = 10000 (your code).

np.random.randint(18, 79, size=10000) # Picks 10,000 numbers at once.

(d) size = (rows, cols) → Get a 2D grid of random number.

→ It's alternate of "for _ in range(1000)" loop provided by Numpy.

(3) 'spends': np.random.normal(loc=50, scale=20, size=10,000).

The create 10,000 random ~~spotting~~ spending amounts that follow a bell curve pattern (normal distribution)

(1) loc=50 ("center" of the bell curve).

→ The average spending amount is $50.

→ Most people will spend around $50.

(2) scale=20 ("width" of the bell curve)

→ About 68% of people spend between 30 and 70 (±$20 from average).

→ About 95% ispend between 10 and 90 (±$40 from average)

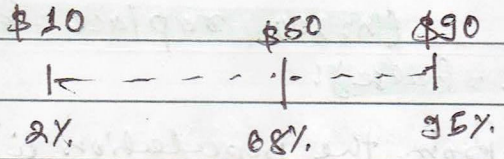
(3) size=10000

→ Creates 10,000 random spending amount at once

→ like printing 10,000 receipts with different amount.

PAGE NO.....
DATE / /

Visual



(3) population.loc [population['age'] < 30, 'spends'] *= 0.7

This line adjusts the spending habits for all people under 30 year old in your dataset by reducing their spending by 30%. Here's a detailed breakdown.

(1) population['age'] < 30:
- a) Creates a True/False mask identifying all rows where age is less than 30.
- b) Example: if someone is 25 -> True, if 35 -> False

(2) Population.loc[mask, 'spends']:
-> Selects ONLY the 'spends' values for people under 30

(3) *= 0.7:
-> Multiplies each selected spending value by 0.7 (30% random)
-> Equivalent to: spends = spends * 0.7.

Before and After Example:

Age	Original Spend	Adjusted Spend (if age < 30).
25	$50	35 ($50 X 0.7)
40	$60	$60 (unchanged)
28	$80	56 ($80 X 0.7)

=> After this line, your population DataFrame no longer follows the original normal distribution, it is transformed with new ~~original~~ normal distribution. It is created for realistic Simulation, Creating Population Heterogeneity and Impact on Statistics.

PAGE NO.....
DATE / /

(4) Random Sampling (unbiased). / random_sample = population.sample (n=200, replace = False).

- sample (n=200, replace = False):

↳ Takes 200 random rows from the population without replacement
↳ Replacement means only selected once. (replace = False).

- random_sample['spends'].mean().

↳ prints the average spending in the random sample.

(5) Biased Sampling (like Literary Digest Poll) - / biased_sample = population[population['age'] > 40].sample (n=200)

↳ Bias: only samples people over 40. Since older people spend more (young spend less), this overestimates the true mean.

↳ Example of sampling bias: Similar to the 1936 Literacy Digest poll that over represented wealthy voters and incorrectly predicted the election.

(6) Stratified Sampling (Age Groups).

(a) strata = pd.cut(population['age'], bins = [18, 30, 50, 70]).

↳ pd.cut() splits the population into 3 age strata: 18-30 (young), (30-50) middle aged, (50, 70) older.

(b) stratified_sample = population.groupby(strata, group_keys = False).apply(lambda x: x.sample(n=50)).

↳ b.(1) → groupby(strata).

↳ Groups the DataFrame by these age categories.

↳ Creates 3 separate sub-DataFrames (one per age group).

↳ b.(2) → group_keys = False

→ Technical parameter that prevents adding extra index levels

Examples: Before (without group keys = False):

Group	Age	Spend.
18-30	25	42.00
18-30	22	38.50

After (with group_keys = False):

Age	Spend.
25	42.00
22	38.50

The data is identical - just without the extra group ~~keys~~ labels in the output.

(4) .apply (lambda x: x.sample(n=50)).

- The magic happens here!

L) lambda x: passes each age group separately

L) x.sample(n=50) takes 50 random people from that age group

→ Does this for all 3 age group independantly.

(5) Final Result:

→ 50 young + 50 middle + 50 older = 150 total samples.

→ Each age group is equally represented (balanced design).

PAGE NO.....
DATE / /

Ch.1 Bias

1. What is Bias?

↳ Bias = Systematic error that pushes results in a consist wrong direction.

↳ Not Random Noise: Unlike random errors (scattered, bias always in the same way.

2) The target Example

Unbiased shooter	Biased shooter
Shots scattered randomly around bullseye	Shots cluster off-center (e.g. upper-right).
Error is random (no pattern)	Error is systematic (consistent drift).

3) Types of Bias in Data Science

(1) Sampling Bias:

↳ Example: Surveying only Twitter users to predict elections → misses non-Twitter voters.

(2) Measurement Bias

↳ Example: A scale that always add +5kg → consistently overestimate weight.

(3) Model Biases

↳ Example: Predicting house prices with only square footage → ignore location, leading to consistent under/over estimation.

4) How to Detect Bias?

↳ Compare result to a ground truth (e.g. known value benchmarks)

↳ check if errors follow a pattern (e.g. always over

PAGE NO.....
DATE / /

Ch 1 Bias

1. What is Bias?

↳ Bias = Systematic error that pushes results in a consistent wrong direction.

↳ Not Random Noise: Unlike random errors (scattered shots), bias is always in the same way.

2) The target Example

Unbiased shooter	Biased shooter
Shots scattered randomly around bullseye	Shots cluster off-center (e.g. upper-right).
Error is random (no pattern)	Error is systematic (consistent drift).

3) Types of Bias in Data Science

(1) Sampling Bias:

↳ Example: Surveying only Twitter users to predict elections → misses non-Twitter voters.

(2) Measurement Bias

↳ Example: A scale that always add +5kg → consistently overestimate weight.

(3) Model Bias

↳ Example: Predicting house prices with only square footage → ignore location, leading to consistent under/overestimation.

4) How to Detect Bias?

↳ Compare result to a ground truth (e.g. known values, benchmarks)

↳ check if errors follow a pattern (e.g. always overestimating for a subgroup).

PAGE NO.....
DATE / /

Python code

Simulate unbiased vs. biased shooters.



```
Import numpy as np.
import matplotlib.pyplot as plt

# Simulate unbiased vs. biased shooters.
np.random.seed(42)

# Unbiased: shots centered at (0,0) with random noise
(1) unbiased_x = np.random.normal(0, 1, 1000) # μ, σ, size
    unbiased_y = np.random.normal(0, 1, 1000) # μ, σ, size.

# Biased: shots centered at (2,2) with same noise.
(2) biased_x = np.random.normal(2, 1, 1000) # μ, σ, size
    biased_y = np.random.normal(2, 1, 1000) # μ, σ, size.

# Plot
plt.figure(figsize=(10,5)).
plt.subplot(1,2,1).  # rows, column, positions.
plt.scatter(unbiased_x, unbiased_y, alpha=0.5).
plt.title("Unbiased Shooter (Random Error)")
plt.axvline(0, c='red'); plt.axhline(0, c='red').

plt.subplot(figsize=(10,5)).# rows, column, positions.
  plt.scatter(biased_x, biased_y, alpha=0.5).
  plt.title("Biased Shooter (systematic + Random Error)")
  plt.axvline(0, c='red'); plt.axhline(0, c='red').
  plt.show()
```

PAGE NO.....
DATE / /

(1) Unbiased
→ Shots are centered at the origin (0,0)
→ Spread follows standard normal distribution (bell curve)
→ Represent pure random error around the bullseye.

(2) Biased Shooter
→ Shots are centered at (2,2).
→ Same random variation ($\sigma = 1$) but with systematic offset.
→ Represents consistent aiming error plus random variation.

"axvline and axhline paradox!!"

PAGE NO.....
DATE / /

Key Takeaways

1.) Bias ≠ Randomness: Bias is a consist error, randomness is scattered.

2) Sources of Bias: Bad sampling, Flawed measurement, or simplified models.

3) Fix it!
- For sampling: Use random/stratified methods
- For models: Add missing variables or recalibrate.

Think like This:

- A biased AI recruiter might always reject resumes from non-Ivy League schools → systematically misses good candidates.
- A fair recruiter makes random mistakes (e.g. overlooks a few good resumes) but no consistent patterns.

dhawanbohring

PAGE NO.....
DATE / /

2. (b) Random Selections.

(1) Why Random Sampling?

- Goal: Avoid bias (like the Literary Digest's 1936 poll failure).
- Key idea: Every member of the population has an equal chance of being selected
- Example:
  - Bad: Survey only rich voters → biased prediction
  - Good: Randomly pick voters from all income groups → accurate predictions.

(2) Challenge in Random Sampling

(a) Defining the Population: Who Really Counts?

problem:

Your "population" might include noise, duplicates, or irrelevants entities that skew results.

key Question to Ask:

↳ Active vs. Inactive: Should past customer (e.g, no purchase in 2 years) be included?

↳ Refunds: Do refunded ordered count as "sales"?

↳ Reseller vs. End User: Are bulk buyer fundamentally different?

↳ Test Data: should internal/QA accounts be excluded?

Example

if analyzing customer satisfaction

☑ Include: Active customer (purchased in last 6 months)

☒ Exclude: Test accounts, refunded orders, resellers.

(b) Sampling from Flows Timing Matters

Problems:

Data collected at different times may represent different behaviors.

Real-World Cases

- Web traffic
  - 10 AM Visitors: likely professional researching during work hours
  - 10 PM Visitors: casual browser or international users.
- Retail Sales
  - Weekday vs. weekend shoppers.
  - Holidays season vs. regular days.

(3) Stratified Sampling

- Use case: When subgroups (strata) differ significantly (e.g. race, age, income).
- How it works?
  1. Split population into strata (e.g. Black, White Hispanic voters)
  2. Randomly sample within each stratum (even if some groups are small).
- Why? Ensure all subgrouped are represented.

PAGE NO.....
DATE / /

Python code

① Random vs. Stratified Sampling

```
import pandas as pd.
import numpy as np.

# Simulate population: 70% white, 20% Black, 10% Hispanic voters.
np.random.seed(42).
population = pd.DataFrame({
    'race': np.random.choice(['White', 'Black', 'Hispanic'],
        size=1000, p=[0.7, 0.2, 0.1]),
    'preference': np.random.choice('CandidateA', 'CandidateB'],
        size=10000).
})
# Random Sample
random_sample = population.sample(n=1000)
print("Random Sample Counts:\n", random_sample['race']
    .value_counts()).

# Stratified sample
stratified_sample = population.groupby('race', group_keys
    =False).apply(lambda
    x: x.sample(n=20))
print("\nStratified Sample Counts:\n", stratified_sample
    ['race'].value_counts())
```

Explaination

① Population Simulation.

→ creates 10,000 simulated voters with
  → 70% white, 20% Black, 10% Hispanic (race distribution)
  → Random candidate preference (50-50 split).

PAGE NO.....
DATE / /

(2) Simulate web traffic data.

```
import pandas as pd.
import numpy as np.
# 1. Create simulated web traffic data.
    np.random.seed(42).
    traffic = pd.DataFrame({
        'timestamp': pd.date_range('2023-01-01', periods=
                                   10,000, freq='h'),
        'visitor_type': np.random.choice(['weekday',
                                   'weekend'], p=[0.7,0.3]
                                   size=10000),
        'page_views': np.random.poisson(3, 10000).
    })
# 2. Perform stratified sampling (corrected version).
stratified_sample = traffic.groupby('visitor_type', group_keys
                                   =False).apply(lambda x: x.
                                   sample(n=500, replace=False))

# Verify the result.
    print("stratified sample counts:").
    print(stratified_sample['visitor_type'].value_counts()).

# Show sample of the data.
    print("Sample of stratified data:").
    print(stratified_sample.head()).
```

DATE / /

(1) 'timestamp': pd.date_range('2023-01-01', periods = 10000, freq='h'),

parameter	Value	Meaning
start	2023-01-01	1st timestamp.
periods	10000	Total number of timestamps to generate.
freq	'h'	Hourly frequency

pd.date_range()

The pd.date_range() function in pandas is a powerful tool for generating sequence of dates and time.

Basic syntax

```
pd.date_range(
    start=None,
    end=None,
    periods=None,
    freq='D',
    tz=None,
    normalize=False,
    name=None,
    closed=None,
    inclusive=None
)
```

Common frequency Aliases

Alias	Description	Example Output
'T' / 'min'	Minute	2023-01-01 00:00:00 →
		2023-01-01 00:01:00
'H'	Hourly	" 00:00:00 →
		" 00:01:00.

'D'	Calendar Day	2023-01-01 -> 2023-01-02.
'B'	Business day	skips weekends.
'W'	Weekly	2023-01-01 - 2023-01-08
'M'	Month end	2023-01-31 - 2023-02-28
'Q'	Quarter end	2023-03-31 -> 2023-06-30.
'A'/'Y'	Year end	2023-12-31 -> 2024-12-31.

Do 'period' is always in hours.?

-> No, periods in pd.date_range() is not always in hours - it's unit-agnostic and works in conjunction with the freq (frequency) parameter you specify.

periods = Number of time units to generate
freq = Duration of each ~~time~~ unit

(2) 'page_views': np.random.poisson(3, 10000)

-> This line generates simulated page view counts using a Poisson distribution, which is perfect for modeling website traffic. Here's a detailed breakdown.

What it does:

- -> Generate 10,000 random page view counts.
- -> Models the no. of visitors per hour.
- -> Uses λ (lambda) = 3 as the average view rate.)

Key characteristics

Parameter	Value	Meaning
lam	3	Average page views per hour
size	10,000	No. of observation.

Size vs Quality (1) Quality-focused sampling

```
import pandas as pd
import numpy as np
from skle.
# Generate a clean population.
np.random.seed(42).
clean_data = pd.DataFrame({
    'feature1': np.random.normal(50, 10, 100000)
    'feature2': np.random.poisson(5, 100000).
    'target': np.random.choice([0,1], p=[0.7, 0.3],
                               size = 100000).
})
# Take a high quality random sample.
sample = clean_data.sample(n=2000, random_state=42)

# Data Quality checks.
def quality_check(df):
    print(f"Missing values: {df.isnull().sum().sum()}")
    print(f"Duplicate rows: {df.duplicate().sum()}")
    print(f"outliers (>3σ): {((df['feature1'] -
                df['feature1'].mean()).abs() >
                3 * df['feature1'].std()).sum()}")

print("--sample Quality--")
quality_checks(sample).
```

(1) Why there is random_state=42?

↳ It act like a "seed" for the random number generator.

⌐) Same seed ⇒ same sequence of "random" numbers.

(2)(a) Does isnull() and duplicate access all columns?

→ Yes they do.

(b) Why is two sum() in df.isnull().sum().sum()

PAGE NO.....
DATE / /

df.isnull() -> This returns a DataFrame of booleans, with [True] for missing (null) values and False otherwise.

1st sum() → is applied column-wise by default. Since [True] is treated as 1 and False as 0. This gives the no. of missing values per column.

2nd sum() → Now you have a series of missing value counts per column. The second [.sum()] add them up to get the total no. of missing values in the entire DataFrames.

(c) Why two sum() is not need for duplicated()?

Example:

	A	B	C
0	1.0	1	NaN
1	2.0	2	2.0
2	NaN	2	3.0
3	4.0	2	4.0

→ Missing values: df.isnull().sum().sum()

Step 1 : df.isnull() → shows [True] where values are missing:

	A	B	C
0	False	False	True
1	False	False	False
2	True	False	False
3	False	False	False

Step 2 sum() (first one) - Adds up missing values per columns.

A	1
B	0
C	1

(3) ((df['feature1'] - df['feature1'].mean())
.abs() > 3 * df['feature1'].std()).sum().

(1) df['feature1'] -> select the column named.
['feature']

(2) df['feature1'].mean() -> calculates the mean (average) of the column.

(3) df['feature1'] - df['feature1'].mean()
-> subtracts the mean from each value in 'feature1' -> gives deviation from the mean for each row.

(4) .abs().
-> Takes the absolute value of the deviations
-> so both high and low outliers are considered.

(5) 3 * df['feature1'].std().
-> calculates 3 times the std, based on the empirical rule.
↳ In normal distribution, ~ 99.7% of values lie with in ± 3σ.
↳ so anything beyond 3σ is considered a potential outliers.

Note: -> df['feature1'].std() = np.std(df['feature1'], ddof=1)
-> It uses ddof=1 by default.

Step 3 . sum() (second one) → Adds column totals.
1 (from A) + 0 (from B) + 1 (from C) = 2.

so, we need two .sum()
↳ first to count missing per column.
↳ second to get the total.

→ Duplicate rows : df.duplicated().sum().

Step 1 Return a single series

0	False
1	False
2	False
3	False

Step 2 Just one .sum() is enough.

→ Because it's already a 1D list (a series) of True/False - one value per row - and we just want the count of True

2(c) Size vs Quality

DATE / /

1. When Smaller Samples Are Better?

- Why?
  - ↳ Easier to ensure data quality (clean missing values, inspect outliers)
  - ↳ More efficient for exploratory analysis
  - ↳ Reduces bias through careful random sampling.

- Use Cases:
  - ↳ Surveys (e.g. political polling)
  - ↳ A/B testing (e.g. webpage conversions)
  - ↳ Training pilot ML models.

2. When Big Data is Necessary

- Why?
  - ↳ Needed for sparse patterns (e.g. rare search queries).
  - ↳ Captures long-tail distributions (e.g. niche user behaviors).

- Use Cases
  - ↳ Search engines (Google, Bing).
  - ↳ Recommendation systems (Netflix, Amazon)
  - ↳ Fraud detection (rare anomalies).

Key difference

Small Data	Big Data
☑ Higher quality control	☑ Captures rare events
☑ Lower computational cost	☒ Requires distributed system
☒ Misses long-tail patterns	☒ Harder to clean (explore.

Used when

- → The population is homogeneous (e.g. voter demographics)
- → modeling rare events (e.g. fraud, niche trends)
- → The signal is in the sparsity (e.g. recommendation system).

PAGE NO.....
DATE / /

1.(1) Sample Mean Vs. Population Mean: Core Concepts

1. Key Definitions

- **Population Mean (μ):**

↳ The true average of the entire population. Often unknown in practice. E.g. Average height of all adults in a country.

Sample Mean ($\bar{x}$):

The average of a subset (sample) taken from the population. Used to estimate μ. Example height of 1,000 randomly surveyed adults.

2) Why the Distinction Matters

- Samples are observed, population are inferred.

↳ We rarely have data for entire populations (too expensive/time-consuming).

↳ Law of Large Numbers: As sample size grows, $\bar{x} \to \mu$,

↳ Central Limit Theorem: sample means follow a normal distribution (even if population doesn't).

Where: μ = Ground truth (population), $\bar{x}$ = Estimate (sample)

3) Common Pitfalls

- **Bias:** if the sample isn't random (e.g. only surveying basketball players) $\bar{x} \neq \mu$ (sample mean ≠ True mean).

- **Small samples:** Higher variability in $\bar{x}$ (less reliable estimate) means small samples are less Reliable. Big sample → Good estimate, Small sample → Unreliable Estimate

PAGE NO.....
DATE / /

Sample Mean vs Population Mean.

```
import numpy as np.

# population (height in cm, normally distributed).
np.random.seed(42).
population_heights = np.random.normal(170, 10, 1_000_000) # 1M people.
μ = population_heights.mean().
print(f"Population mean (μ): {μ:.2f} cm")

# Sample (n = 1000).
sample = np.random.choice(population_heights,
                          size=1000, replace=False).
x̄ = sample.mean().
print(f"Sample mean (x̄): {x̄:.2f} cm | Error:
      {abs(x̄ - μ):.2f} cm")
```

output

Population mean (μ): 169.98 cm

Sample mean (x̄): 169.53 cm | Error: 0.46 cm

random.sample() vs random.choice()

Feature	np.random.sample()	np.random.choice()
Input Type	Python sequence only	Sequences + Numpy arrays.
Replacement	Always False (no duplicates)	Configurable (True/False).
output Type	List	Numpy array.
Speed	Slower for big data	Faster (optimized for Numpy).
Weights	Not supported	Support probability weights (p)
Error Handling	Raises error if K > N	Raises error if size > N & replace = False.